



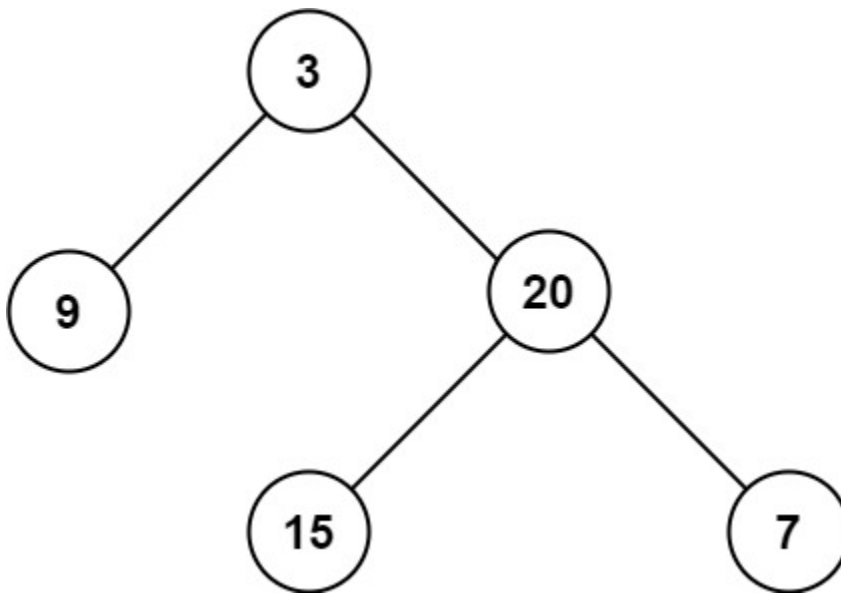
Height Of A Tree

<https://leetcode.com/problems/maximum-depth-of-binary-tree/>

Given the `root` of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: 3

Example 2:

Input: root = [1,null,2]

Output: 2

```

class Solution {
    int dfs(TreeNode* node){
        if (node == nullptr){
            return 0;
        }
        return 1 + max(dfs(node->left) , dfs(node -> right));
    }
public:
    int maxDepth(TreeNode* root) {
        return dfs(root);
    }
};

```

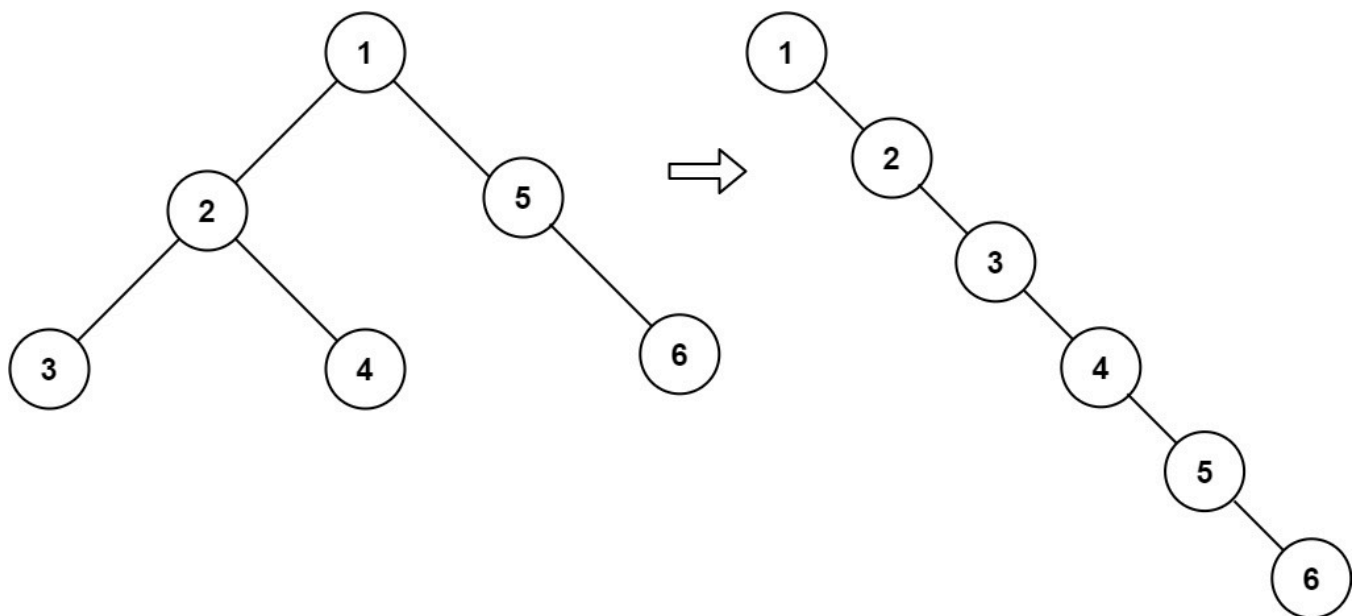
Check If the Binary tree is balanced or not.

<https://leetcode.com/problems/flatten-binary-tree-to-linked-list/>

Given the `root` of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the `right` child pointer points to the next node in the list and the `left` child pointer is always `null`.
- The "linked list" should be in the same order as a **pre-order traversal** of the binary tree.

Example 1:



Input: `root = [1,2,5,3,4,null,6]`

Output: [1,null,2,null,3,null,4,null,5,null,6]

Example 2:

Input: root = []

Output: []

Example 3:

Input: root = [0]

Output: [0]

```
class Solution {
    int depth(TreeNode* node){
        if (node == nullptr) return 0;
        return 1 + max(depth(node->left),depth(node->right));
    }
public:
    bool isBalanced(TreeNode* root) {
        if (root == nullptr){
            return true;
        }
        bool cur = abs(depth(root->left) - depth(root->right)) <= 1;
        return cur & isBalanced(root->left) & isBalanced(root->right);
    }
};
```

Diameter of a binary tree

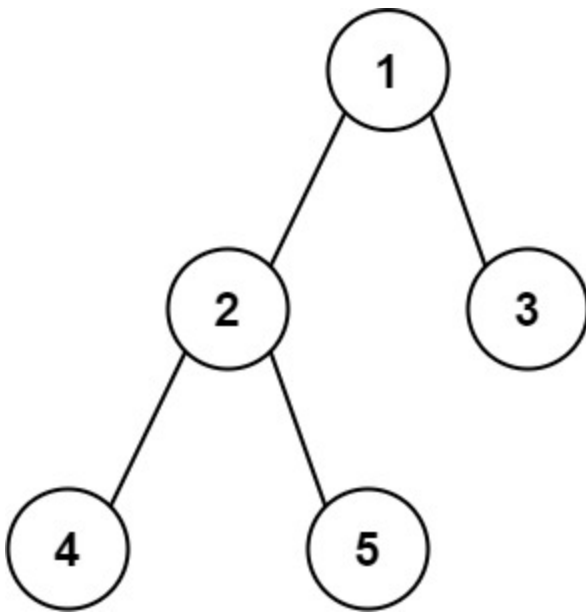
<https://leetcode.com/problems/diameter-of-binary-tree/>

Given the `root` of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the `root`.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

```
class Solution {
    int diameter = 0;
    int depth(TreeNode* root){
        if (root == nullptr) return 0;
        int left = depth(root->left);
        int right = depth(root->right);
        diameter = max(diameter, left + right);
        return 1 + max(left,right);
    }
public:
    int diameterOfBinaryTree(TreeNode* root) {
        if (root == nullptr) return 0;
        int i = depth(root);
        return diameter;
    }
};
```

Binary Tree Maximum Path Sum

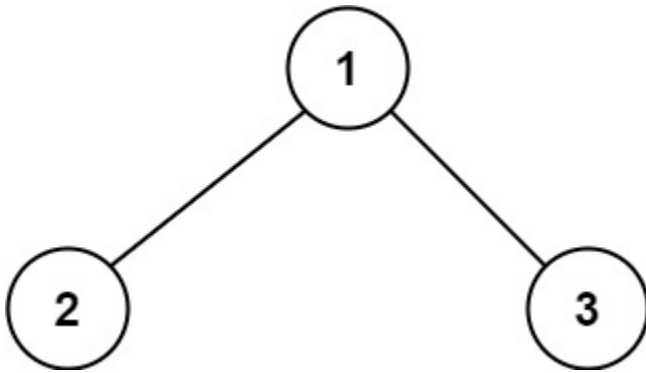
<https://leetcode.com/problems/binary-tree-maximum-path-sum/>

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

Given the **root** of a binary tree, return *the maximum path sum of any non-empty path*.

Example 1:

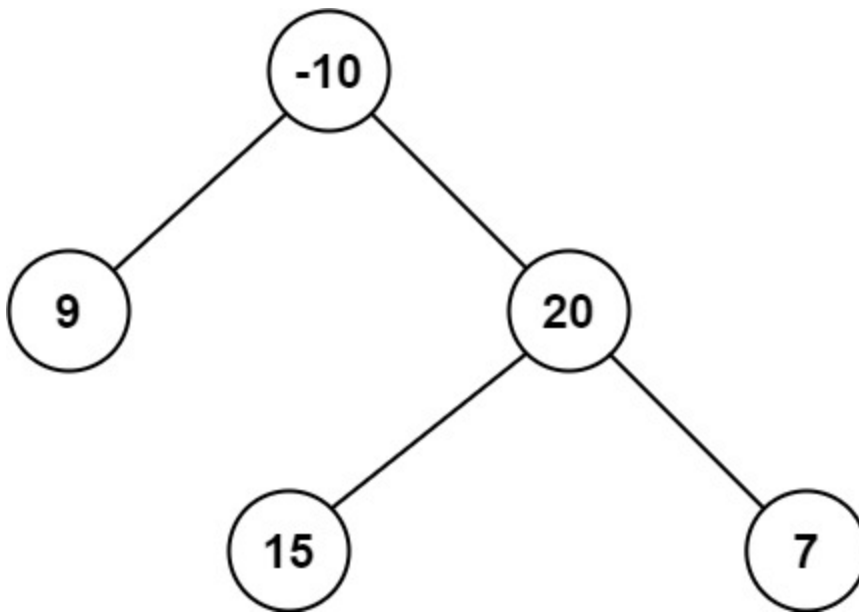


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of $15 + 20 + 7 = 42$.

```
class Solution {
    int maxPath = INT_MIN;
    int pathSum(TreeNode* root){
        if (root == nullptr) return 0;
        int leftSum = max(0, pathSum(root->left));
        int rightSum = max(0, pathSum(root->right));
        int ans = leftSum + rightSum + root->val;
        maxPath = max(maxPath, ans);
        return root->val + max(leftSum, rightSum);
    }
public:
    int maxPathSum(TreeNode* root) {
        int i = pathSum(root);
        return maxPath;
    }
};
```

Check if the given Tree is same

<https://leetcode.com/problems/same-tree/>

```

class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == nullptr & q == nullptr) return true;
        if (p == nullptr | q == nullptr) return false;
        bool cur = p->val == q->val;
        return cur & isSameTree(p->left,q->left) & isSameTree(p->right,q->right);
    }
};

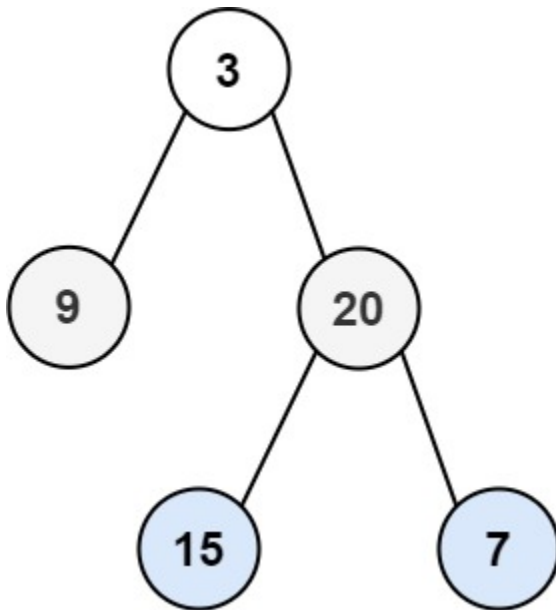
```

Binary Tree Zigzag traversal

<https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/>

Given the `root` of a binary tree, return *the zigzag level order traversal of its nodes' values*.
(i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: 3,[20,9],[15,7]

Example 2:

Input: root = [1]

Output: 1

Example 3:

Input: root = []

Output: []

```
class Solution {
    int depth(TreeNode* root){
        if (root == nullptr) return 0;
        return 1 + max(depth(root->left),depth(root->right));
    }
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> traversals;
        queue<TreeNode*> toVisit;
        toVisit.push(root);
        for (int i = 0; i < depth(root); i++){
            queue<TreeNode*> nextVisit;
            vector<int> traversal;
            while (!toVisit.empty()){
                TreeNode* cur = toVisit.front();
                traversal.push_back(cur->val);
                toVisit.pop();
                if (cur->left != nullptr) nextVisit.push(cur->left);
                if (cur->right != nullptr) nextVisit.push(cur->right);
            }
            toVisit = nextVisit;
            if (traversal.size() == 0) continue;
            if (i%2) reverse(traversal.begin(),traversal.end());
            traversals.push_back(traversal);
        }
        return traversals;
    }
};
```

Boundary Traversal Of a Binary Tree

<https://leetcode.com/problems/boundary-of-binary-tree/>

Given a Binary Tree, find its Boundary Traversal. The traversal should be in the following order:

1. **Left boundary nodes:** defined as the path from the root to the left-most node ie- the leaf node you could reach when you always travel preferring the left subtree over the right subtree.
2. **Leaf nodes:** All the leaf nodes except for the ones that are part of left or right boundary.
3. **Reverse right boundary nodes:** defined as the path from the right-most node to the root. The right-most node is the leaf node you could reach when you always travel preferring the right subtree over the left subtree. Exclude the root from this as it was already included in the traversal of left boundary nodes.

Note: If the root doesn't have a left subtree or right subtree, then the root itself is the left or right boundary.

Example 1:

Input:

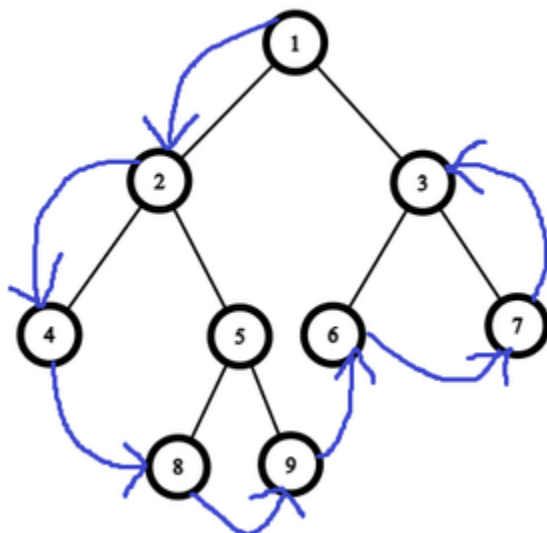
```

1
/
2 3
/\ /\
4 5 6 7
/
8 9

```

Output: 1 2 4 8 9 6 7 3

Explanation:



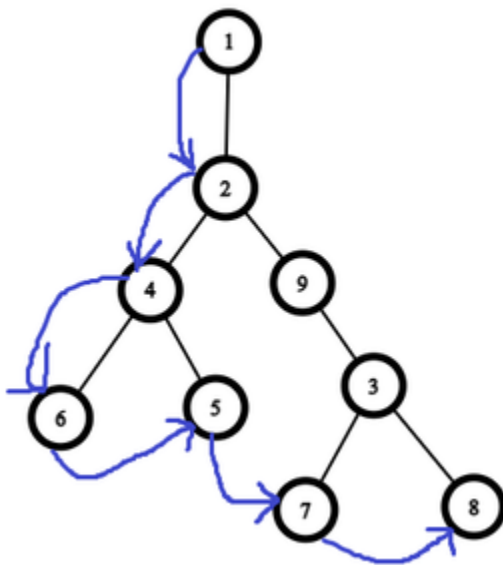
Example 2:

Input:

```
1
/
2
/
4 9
/\
6 5 3
/
7 8
```

Output: 1 2 4 6 5 7 8

Explanation:



As you can see we have not taken the right subtree.

Y**our Task:**

This is a function problem. You don't have to take input. Just complete the **function boundary()** that takes the root node as input and returns an array containing the boundary values in anti-clockwise.

```

class Solution {
    vector<int> path;
    void L(Node* node) {
        if (!node || (!node->left && !node->right)) return;
        path.push_back(node->data);
        if (node->left) L(node->left);
        else L(node->right);
    }

    void leaf(Node* node) {
        if (!node) return;
        if (!node->left && !node->right) {
            path.push_back(node->data);
            return;
        }
        leaf(node->left);
        leaf(node->right);
    }

    void R(Node* node) {
        if (!node || (!node->left && !node->right)) return;
        if (node->right) R(node->right);
        else R(node->left);
        path.push_back(node->data); // add after child visit(reverse)
    }
public:
    vector<int> boundary(Node* root) {
        if (!root) return {};
        path.push_back(root->data);

        // Left Boundary
        if (root->left) L(root->left);

        // Leaf Nodes
        leaf(root->left);
        leaf(root->right);

        // Right Boundary
        if (root->right) R(root->right);
    }
};

```

```
    return path;
  }
};
```

Vertical Order Traversal of a binary tree

<https://leetcode.com/problems/vertical-order-traversal-of-a-binary-tree/>

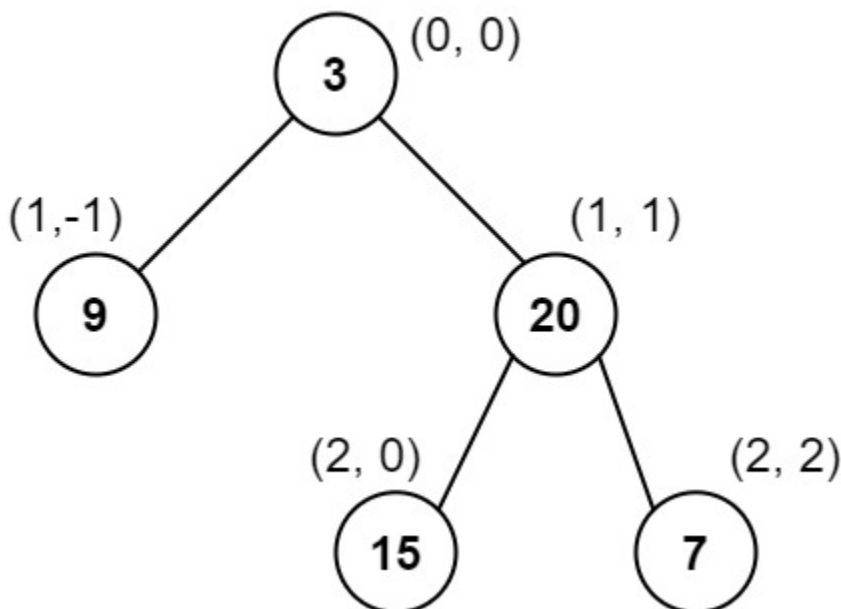
Given the `root` of a binary tree, calculate the **vertical order traversal** of the binary tree.

For each node at position (row, col) , its left and right children will be at positions $(row + 1, col - 1)$ and $(row + 1, col + 1)$ respectively. The root of the tree is at $(0, 0)$.

The **vertical order traversal** of a binary tree is a list of top-to-bottom orderings for each column index starting from the leftmost column and ending on the rightmost column. There may be multiple nodes in the same row and same column. In such a case, sort these nodes by their values.

Return the **vertical order traversal** of the binary tree.

Example 1:



Input: `root = [3,9,20,null,null,15,7]`

Output: `[9],[3,15],[20],[7]`

Explanation:

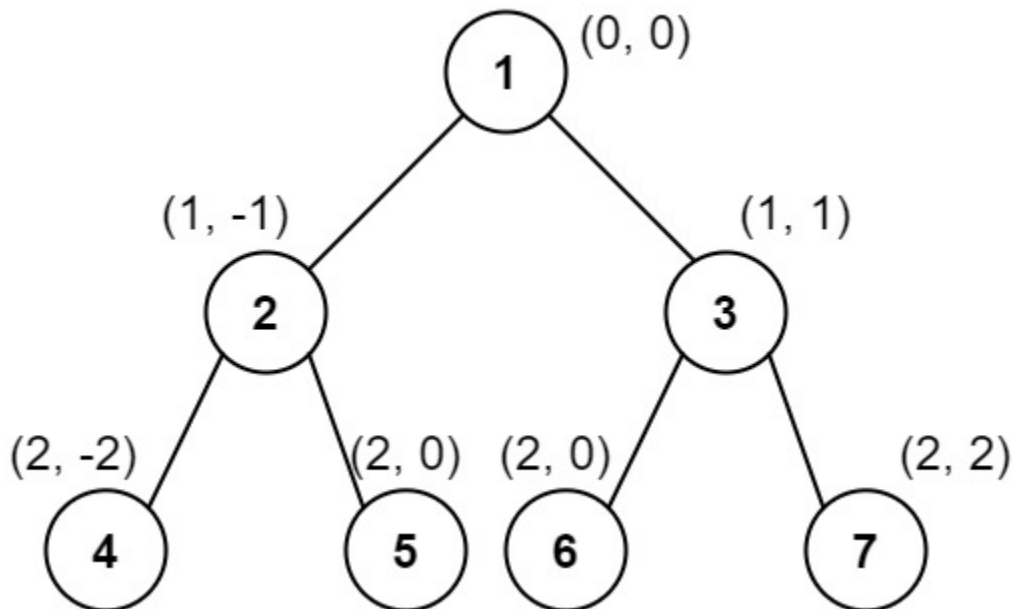
Column -1: Only node 9 is in this column.

Column 0: Nodes 3 and 15 are in this column in that order from top to bottom.

Column 1: Only node 20 is in this column.

Column 2: Only node 7 is in this column.

Example 2:



Input: root = [1,2,3,4,5,6,7]

Output: 4],[2],[1,5,6],[3],[7

Explanation:

Column -2: Only node 4 is in this column.

Column -1: Only node 2 is in this column.

Column 0: Nodes 1, 5, and 6 are in this column.

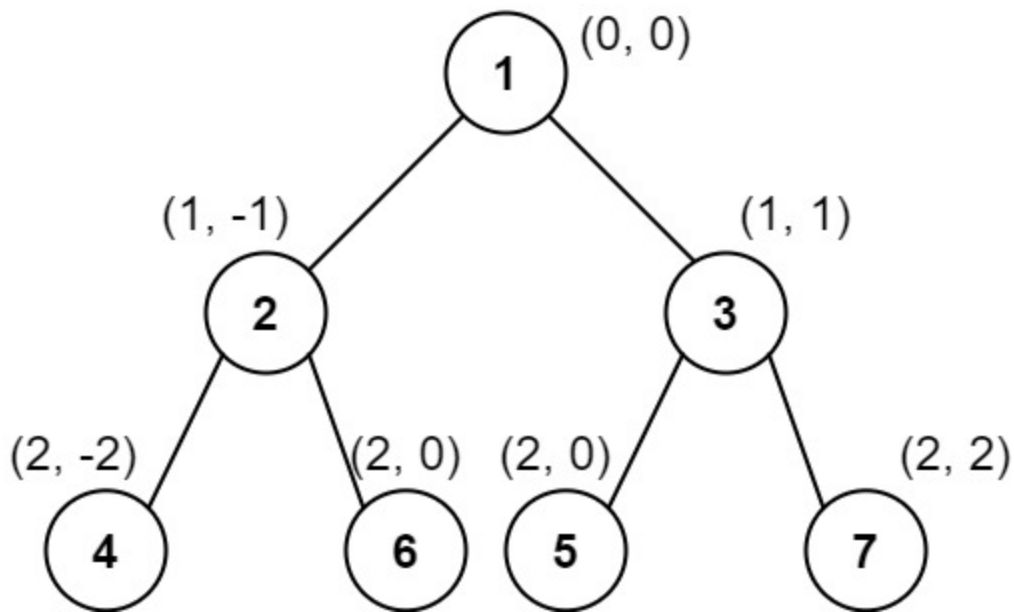
1 is at the top, so it comes first.

5 and 6 are at the same position (2, 0), so we order them by their value, 5 before 6.

Column 1: Only node 3 is in this column.

Column 2: Only node 7 is in this column.

Example 3:



Input: root = [1,2,3,4,6,5,7]

Output: 4,[2],[1,5,6],[3],[7]

Explanation:

This case is the exact same as example 2, but with nodes 5 and 6 swapped.

Note that the solution remains the same since 5 and 6 are in the same location and should be ordered by their values.

```

class Solution {
private:
    map<int, map<int, multiset<int>>> columnMap;

    void dfs(TreeNode* root, int row, int col) {
        if (!root) return;

        columnMap[col][row].insert(root->val);

        dfs(root->left, row + 1, col - 1);
        dfs(root->right, row + 1, col + 1);
    }

public:
    vector<vector<int>> verticalTraversal(TreeNode* root) {
        dfs(root, 0, 0);

        vector<vector<int>> result;
        for (const auto& column : columnMap) {
            vector<int> columnValues;
            for (const auto& rowMap : column.second) {
                columnValues.insert(columnValues.end(), rowMap.second.begin(), rowMap.second.end());
            }
            result.push_back(columnValues);
        }

        return result;
    }
};

```

Top View Of a binary Tree

<https://www.geeksforgeeks.org/problems/top-view-of-binary-tree/1>

Given below is a binary tree. The task is to print the top view of binary tree. Top view of a binary tree is the set of nodes visible when the tree is viewed from the top. For the given below tree

```

    /  \
   2    3
  / \  / \
 4  5 6  7

```

Top view will be: 4 2 1 3 7

Note: Return nodes from **leftmost** node to **rightmost** node. Also if 2 nodes are outside the shadow of the tree and are at same position then consider the left ones only(i.e. leftmost).

For ex - **1 2 3 N 4 5 N 6 N 7 N 8 N 9 N N N N N** will give **8 2 1 3** as answer. Here 8 and 9 are on the same position but 9 will get shadowed.

Example 1:

Input:

```

  1
 /
2  3

```

Output: 2 1 3

Example 2:

Input:

```

  10
 /
20   30
/ \  /
40 60 90 100

```

Output: 40 20 10 30 100

Your Task:

Since this is a function problem. You don't have to take input. Just complete the function **topView()** that takes **root node** as parameter and returns a list of nodes visible from the top view from left to right.


```

class Solution
{
public:
    //Function to return a list of nodes visible from the top view
    //from left to right in Binary Tree.
    vector<int> topView(Node *root)
    {
        //Your code here
        if (root == nullptr) return {};
        Node* node = root;
        map<int,int> topMost;
        queue<pair<Node*,int>> toVisit;
        toVisit.push({node,0});
        while (!toVisit.empty()){
            auto cur = toVisit.front();
            toVisit.pop();
            if (topMost.find(cur.second) == topMost.end()){
                topMost[cur.second] = cur.first->data;
            }
            if (cur.first->left != nullptr) toVisit.push({cur.first->left,cur.second - 1});
            if (cur.first->right != nullptr) toVisit.push({cur.first->right, cur.second + 1});
        }
        vector<int> ans;
        for (auto it: topMost){
            ans.push_back(it.second);
        }
        return ans;
    }
};

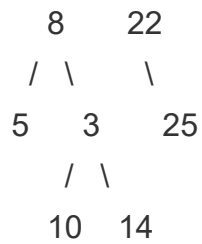
```

Bottom view of a binary tree

<https://www.geeksforgeeks.org/problems/bottom-view-of-binary-tree/1>

Given a binary tree, print the bottom view from left to right.

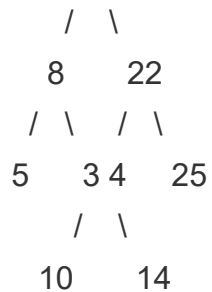
A node is included in bottom view if it can be seen when we look at the tree from bottom.



For the above tree, the bottom view is 5 10 3 14 25.

If there are **multiple** bottom-most nodes for a horizontal distance from root, then print the later one in level traversal. For example, in the below diagram, 3 and 4 are both the bottommost nodes at horizontal distance 0, we need to print 4.

20



For the above tree the output should be 5 10 4 14 25.

```

class Solution {
public:
    vector<int> bottomView(Node *root) {
        // Your Code Here
        if (root == nullptr) return {};
        Node* node = root;
        map<int,int> topMost;
        queue<pair<Node*,int>> toVisit;
        toVisit.push({node,0});
        while (!toVisit.empty()){
            auto cur = toVisit.front();
            toVisit.pop();

            topMost[cur.second] = cur.first->data;

            if (cur.first->left != nullptr) toVisit.push({cur.first->left,cur.second - 1});
            if (cur.first->right != nullptr) toVisit.push({cur.first->right, cur.second + 1});
        }
        vector<int> ans;
        for (auto it: topMost){
            ans.push_back(it.second);
        }
        return ans;
    }
};

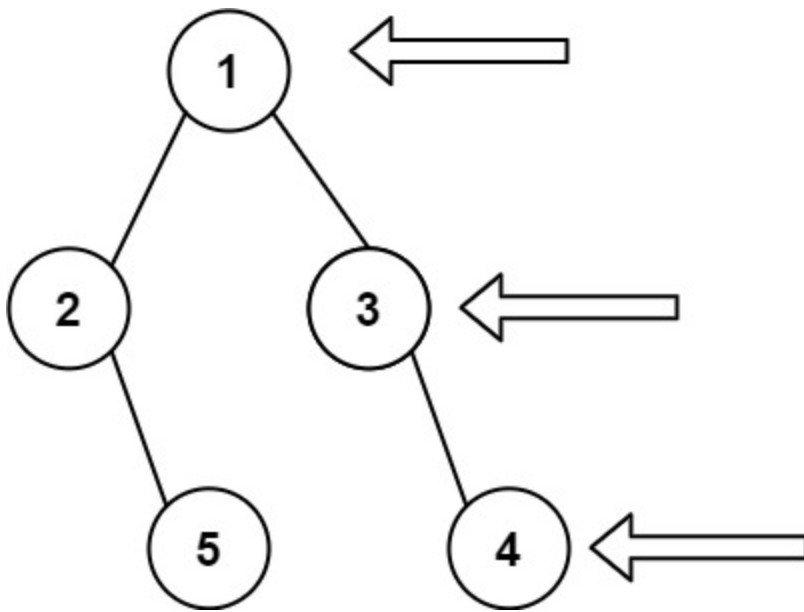
```

Right Side View Of a Binary Tree

<https://leetcode.com/problems/binary-tree-right-side-view/description/>

Given the `root` of a binary tree, imagine yourself standing on the **right side** of it, return *the values of the nodes you can see ordered from top to bottom*.

Example 1:



Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Example 2:

Input: root = [1,null,3]

Output: [1,3]

Example 3:

Input: root = []

Output: []

```

class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        if (root == nullptr) return {};
        TreeNode* node = root;
        map<int,int> topMost;
        queue<pair<TreeNode*,int>> toVisit;
        toVisit.push({node,0});
        while (!toVisit.empty()){
            auto cur = toVisit.front();
            toVisit.pop();
            topMost[cur.second] = cur.first->val;
            if (cur.first->left != nullptr) toVisit.push({cur.first->left,cur.second + 1});
            if (cur.first->right != nullptr) toVisit.push({cur.first->right, cur.second + 1});
        }
        vector<int> ans;
        for (auto it: topMost){
            ans.push_back(it.second);
        }
        return ans;
    }
};

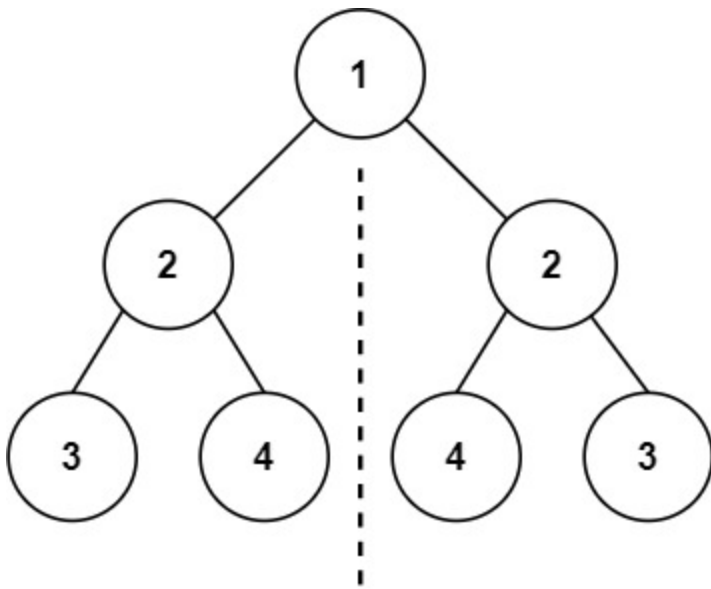
```

Is the binary tree symmetric

<https://leetcode.com/problems/symmetric-tree/description/>

Given the `root` of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

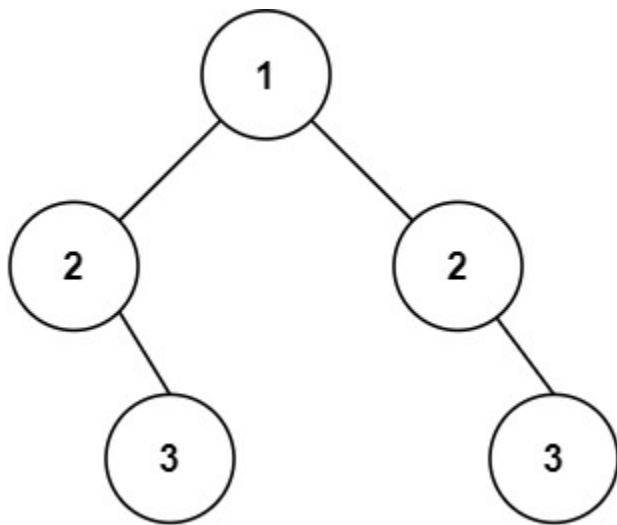
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]

Output: false

```

class Solution {
private:
    bool isPalindrome(vector<int> &v){
        int n = v.size();
        int i = 0, j = n-1;
        while (i < j){
            if(v[i] != v[j]) return false;
            i++,j--;
        }
        return true;
    }
public:
    bool isSymmetric(TreeNode* root) {
        vector<int> trav;
        queue<TreeNode*> toVisit;
        toVisit.push(root);
        while (!toVisit.empty()){
            queue<TreeNode*> newVisit;
            vector<int> v;
            while (!toVisit.empty()){
                TreeNode* cur = toVisit.front();
                toVisit.pop();
                if (cur == nullptr) v.push_back(-101);
                else{
                    v.push_back(cur->val);
                    newVisit.push(cur->left);
                    newVisit.push(cur->right);
                }
            }
            if (!isPalindrome(v)) return false;
            toVisit = newVisit;
        }
        return true;
    }
};

```

Hard?

Root to Node path in a binary tree

<https://www.geeksforgeeks.org/problems/root-to-leaf-paths/1>

Given a **Binary Tree** of nodes, you need to find **all the possible paths** from the **root node** to all the **leaf nodes** of the binary tree.

Example 1:

Input:

```
1
/
2 3
```

Output: 1 2

1 3

Explanation:

All possible paths:

1->2

1->3

Example 2:

Input:

```
10
/
20 30
/
40 60
```

Output: 10 20 40

10 20 60

10 30

Your Task:

Your task is to complete the function **Paths()** which takes the root node as an argument and returns all the possible paths.


```

class Solution {
private:
    void dfs(Node* node, vector<int> &path, vector<vector<int>> &allPaths){
        if (node == nullptr){
            return;
        }
        path.push_back(node->data);
        if (node->left == nullptr and node->right == nullptr) {
            allPaths.push_back(path);
        }
        else {
            dfs(node->left, path, allPaths);
            dfs(node->right, path, allPaths);
        }
        path.pop_back();
    }
public:
    vector<vector<int>> Paths(Node* root) {
        // code here
        vector<vector<int>> allPaths;
        vector<int> path;
        dfs(root, path, allPaths);
        return allPaths;
    }
};

```

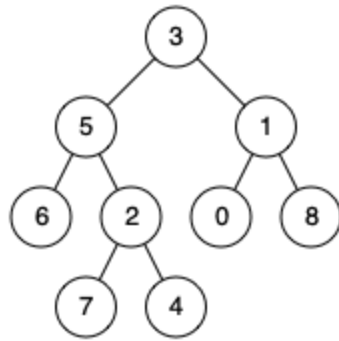
Lowest Common Ancestor of a binary tree

<https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/>

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes **p** and **q** as the lowest node in **T** that has both **p** and **q** as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

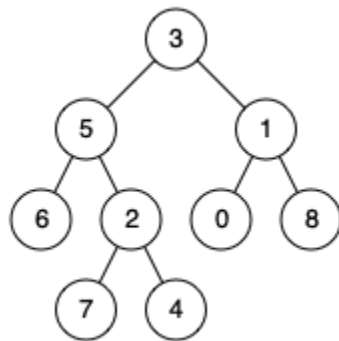


Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

```

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (!root || root == p || root == q) return root;
        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);
        return !left ? right : !right ? left : root;
    }
};

```

Maximum Width of a binary tree

<https://leetcode.com/problems/maximum-width-of-binary-tree/description/>

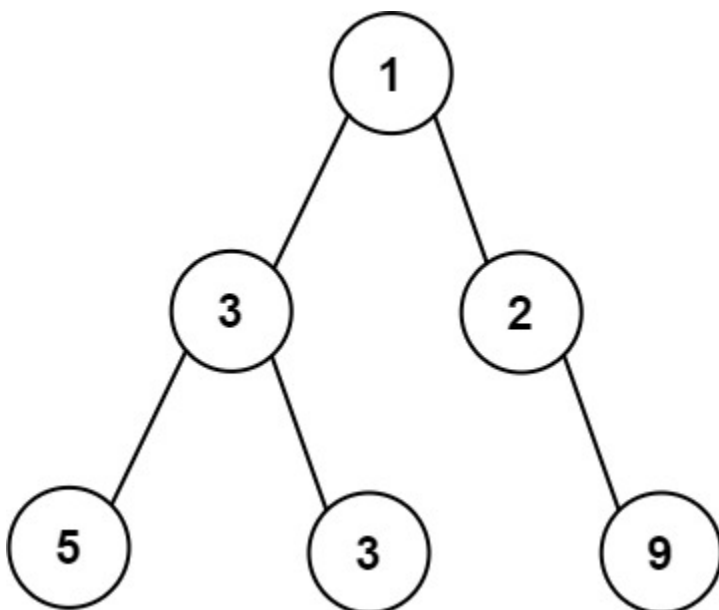
Given the `root` of a binary tree, return the **maximum width** of the given tree.

The **maximum width** of a tree is the maximum **width** among all levels.

The **width** of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is **guaranteed** that the answer will in the range of a **32-bit** signed integer.

Example 1:

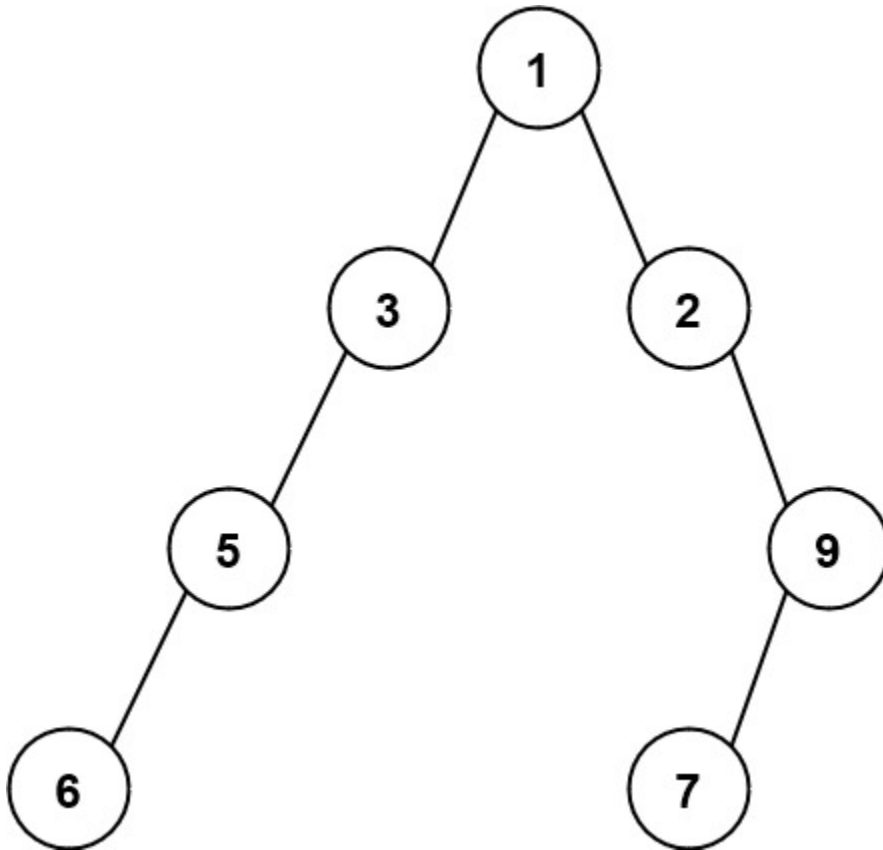


Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:

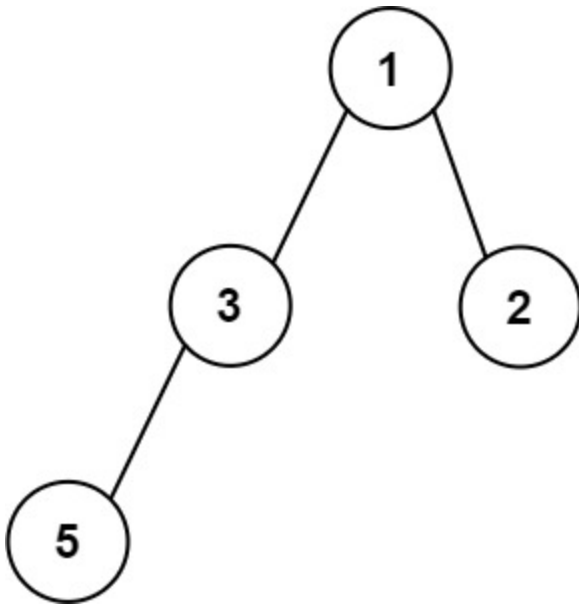


Input: root = [1,3,2,5,null,null,9,6,null,7]

Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).

Example 3:



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

```
class Solution:
    def widthOfBinaryTree(self, root: TreeNode) -> int:
        ans = 0
        if not root:
            return ans

        queue = collections.deque([(root, 0)])
        while queue:
            l, r = queue[0][1], queue[-1][1]
            ans = max(ans, r-l + 1)

            for _ in range(len(queue)):
                node, idx = queue.popleft()
                if node.left:
                    queue.append((node.left, idx*2+1))
                if node.right:
                    queue.append((node.right, idx*2+2))

            return ans
```

Check for Children Sum Property in a Binary Tree

<https://www.geeksforgeeks.org/problems/children-sum-parent/1>

Given a binary tree having **n** nodes. Check whether all of its nodes have the value equal to the sum of their child nodes. Return 1 if all the nodes in the tree satisfy the given properties, else it return 0.

For every node, data value must be equal to the sum of data values in left and right children. Consider data value as 0 for NULL child. Also, leaves are considered to follow the property.

Example 1:

****Input:**

****Binary tree**

35

/

20 15

/ \ \

15 5 10 5

Output: 1

Explanation: Here, every node is sum of its left and right child.

Example 2:

****Input:**

****Binary tree**

1

/

4 3

/

5

Output: 0

Explanation: Here, 1 is the root node and 4, 3 are its child nodes. $4 + 3 = 7$ which is not equal to the value of root node. Hence, this tree does not satisfy the given condition.

```

class Solution{
public:
    //Function to check whether all nodes of a tree have the value
    //equal to the sum of their child nodes.
    int isSumProperty(Node *root)
    {
        if (!root) return 1;
        int child_sum = 0;
        if (!root->left and !root->right) return isSumProperty(root->left) & isSumProperty(root->right);
        if (root->left != nullptr) child_sum += root->left->data;
        if (root->right != nullptr) child_sum += root->right->data;
        return (root->data == child_sum) & isSumProperty(root->left) & isSumProperty(root->right);
    }
};

```

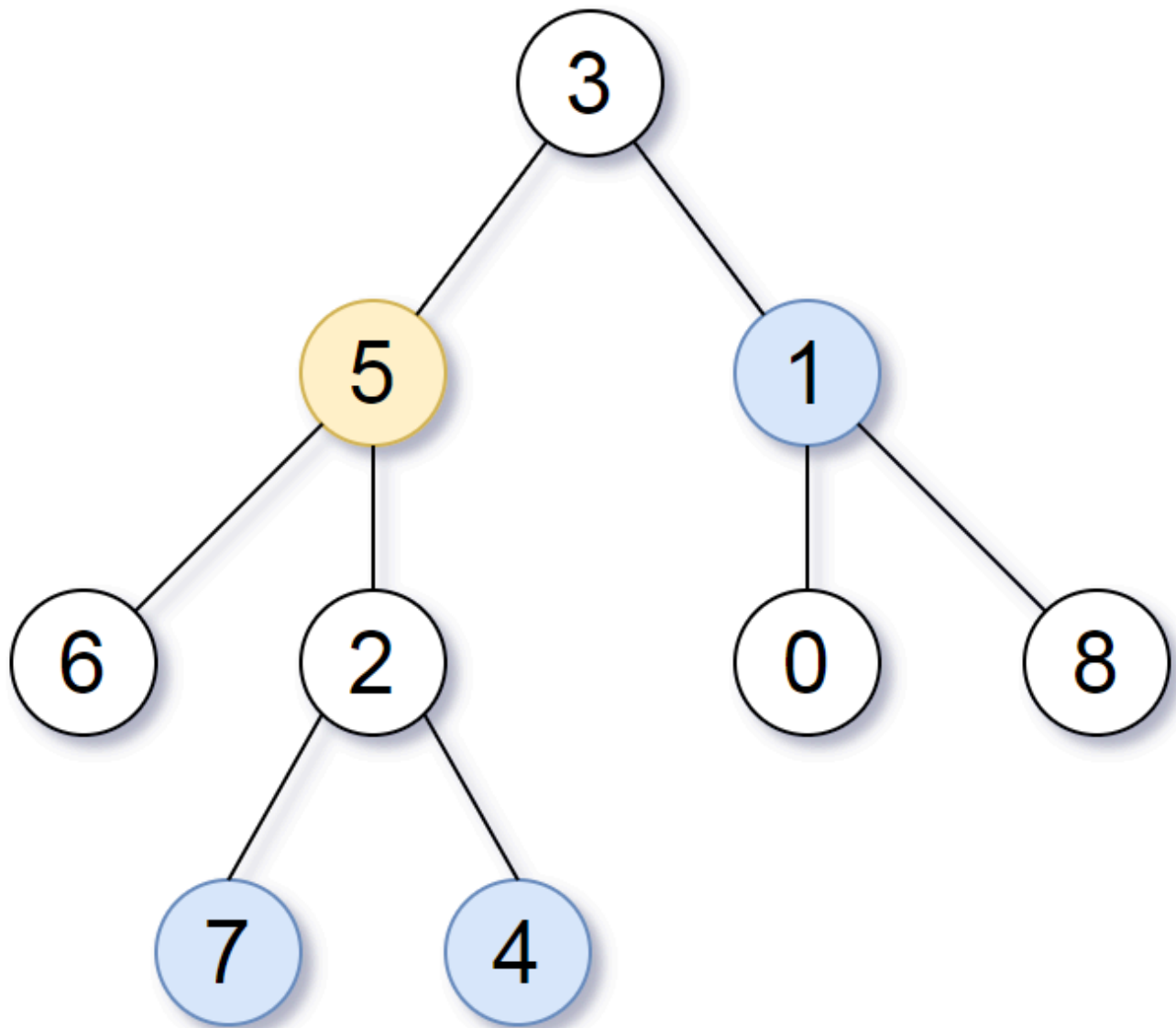
All nodes at a distance K from the target node

<https://leetcode.com/problems/all-nodes-distance-k-in-binary-tree/description/>

Given the `root` of a binary tree, the value of a target node `target`, and an integer `k`, return *an array of the values of all nodes that have a distance `k` from the target node*.

You can return the answer in **any order**.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

Input: root = [1], target = 1, k = 3

Output: []


```

class Solution {
private:
    // Helper function to create a map of parent pointers
    void mapParents(TreeNode* node, map<TreeNode*, TreeNode*>& parent) {
        if (!node) return;
        if (node->left) {
            parent[node->left] = node;
            mapParents(node->left, parent);
        }
        if (node->right) {
            parent[node->right] = node;
            mapParents(node->right, parent);
        }
    }
public:
    vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
        // Edge case: If the tree is empty
        if (!root) return {};
        // Map to store parent pointers
        map<TreeNode*, TreeNode*> parent;
        parent[root] = nullptr;
        mapParents(root, parent);
        // Queue for BFS
        queue<TreeNode*> q;
        q.push(target);
        // Set to keep track of visited nodes
        unordered_set<TreeNode*> visited;
        visited.insert(target);
        int current_level = 0;
        // Perform BFS
        while (!q.empty()) {
            int size = q.size();
            if (current_level == k) break;
            current_level++;
            for (int i = 0; i < size; i++) {
                TreeNode* node = q.front();
                q.pop();
                if (node->left && visited.find(node->left) == visited.end()) {
                    visited.insert(node->left);
                }
            }
        }
    }
};

```

```

        q.push(node->left);
    }
    if (node->right && visited.find(node->right) == visited.end()) {
        visited.insert(node->right);
        q.push(node->right);
    }
    if (parent[node] && visited.find(parent[node]) == visited.end()) {
        visited.insert(parent[node]);
        q.push(parent[node]);
    }
}
}
// Collect all nodes at distance k
vector<int> result;
while (!q.empty()) {
    TreeNode* node = q.front();
    q.pop();
    result.push_back(node->val);
}
return result;
}
};

```

Time to burn the whole tree from a certain node

<https://www.geeksforgeeks.org/problems/burning-tree/1>

Given a binary tree and a **node data** called **target**. Find the minimum time required to burn the complete binary tree if the target is set on fire. It is known that in 1 second all nodes connected to a given node get burned. That is its left child, right child, and parent.

Note: The tree contains unique values.

Examples :

Input:

```

1
/
2 3
/\

```

4 5 6

/ \

7 8 9

10

Target Node = 8

Output: 7

Explanation: If leaf with the value
8 is set on fire.

After 1 sec: 5 is set on fire.

After 2 sec: 2, 7 are set to fire.

After 3 sec: 4, 1 are set to fire.

After 4 sec: 3 is set to fire.

After 5 sec: 6 is set to fire.

After 6 sec: 9 is set to fire.

After 7 sec: 10 is set to fire.

It takes 7s to burn the complete tree.

Input:

1

/

2 3

/ \

4 5 7

//

8 10

Target Node = 10

Output: 5

```

class Solution {
public:
    int minTime(Node* root, int target)
    {
        // Your code goes here
        // create adj list
        map<Node*, vector<Node*>> adj;
        queue<Node*> toVisit;
        toVisit.push(root);
        Node* targ = nullptr;
        while (!toVisit.empty()){
            Node* cur = toVisit.front();
            if (cur->data == target) targ = cur;
            toVisit.pop();
            if (cur->left != nullptr){
                adj[cur].push_back(cur->left);
                adj[cur->left].push_back(cur);

                toVisit.push(cur->left);
            }
            if (cur->right != nullptr){
                adj[cur].push_back(cur->right);
                adj[cur->right].push_back(cur);

                toVisit.push(cur->right);
            }
        }
        queue<pair<Node*,int>> burn;
        burn.push({targ,0});
        int max_time = INT_MIN;
        map<Node*,int> visited;
        while (!burn.empty()){
            auto cur = burn.front();
            burn.pop();
            int cur_time = cur.second;
            Node* cur_node = cur.first;
            visited[cur_node] = 1;
            max_time = max(max_time,cur_time);
            for (Node* i: adj[cur_node]){

```

```

        if (!visited[i]){
            burn.push({i, cur_time+1});
        }
    }
}
return max_time;
}
};

```

Count total nodes in a binary tree

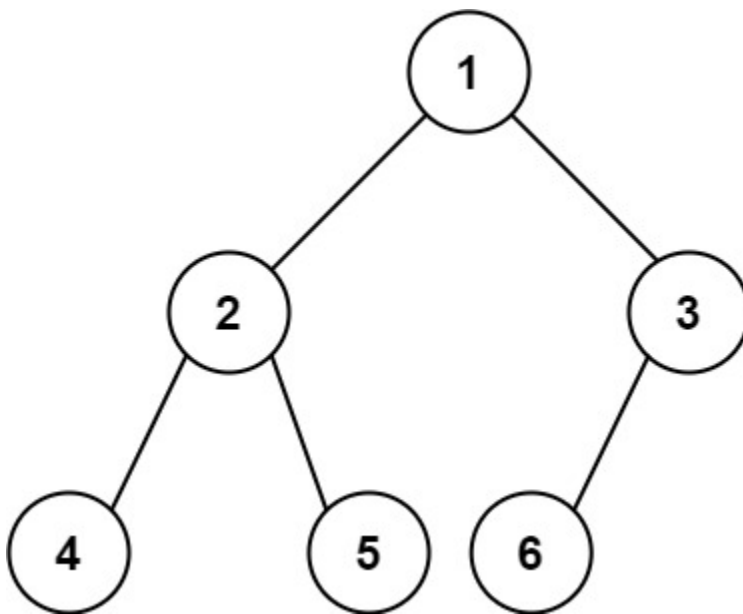
<https://leetcode.com/problems/count-complete-tree-nodes/description/>

Given the **root** of a **complete** binary tree, return the number of the nodes in the tree.

According to [Wikipedia](#), every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between 2^{h-1} and $2^h - 1$ nodes inclusive at the last level h .

Design an algorithm that runs in less than $O(n)$ time complexity.

Example 1:



Input: root = [1,2,3,4,5,6]

Output: 6

Example 2:

Input: root = []

Output: 0

Example 3:

Input: root = [1]

Output: 1

```
class Solution {
public:
    int countNodes(TreeNode* root) {
        if (!root) return 0;
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

Unique binary tree requirements

<https://www.geeksforgeeks.org/problems/unique-binary-tree-requirements/1>

Geek wants to know the traversals required to construct a **unique binary tree**. Given a pair of traversal, return **true** if it is possible to construct unique binary tree from the given traversals otherwise return **false**.

Each traversal is represented with an integer: preorder - 1, inorder - 2, postorder - 3.

Example 1:

Input:

a = 1, b=2

Output: 1

Explanation: We can construct binary tree using inorder traversal and preorder traversal.

Example 2:

Input: a = 1, b=3

Output: 0

Explanation: We cannot construct binary tree using preorder traversal and postorder

traversal.

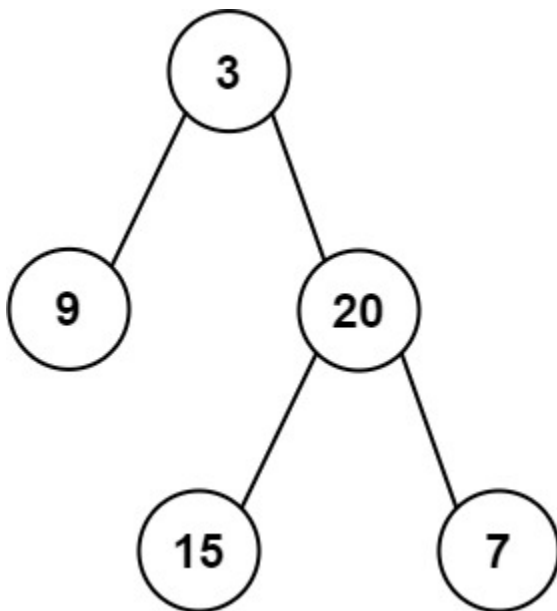
```
class Solution
{
public:
    bool isPossible(int a,int b)
    {
        return ((a+b)%2);
    }
};
```

Construct binary tree from Inorder and PreOrder

<https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/description/>

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the* binary tree.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]

Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]

Output: [-1]

```
```cpp
class Solution {
public:
 TreeNode* constructTree(vector<int> & preorder, int preStart, int preEnd, vector<int> & inorder,
 int inStart, int inEnd) {
 if ((preStart > preEnd) || (inStart > inEnd))
 return NULL;
 TreeNode* root = new TreeNode(preorder[preStart]);
 int elem = indexes[preorder[preStart]]; //location of root in inOrder
 int nElem = elem - inStart; //number of elem in the left subTree
 root->left = constructTree(preorder, preStart+1, preStart+nElem, inorder, inStart, elem - 1,
 indexes);
 root->right = constructTree(preorder, preStart + nElem + 1, preEnd, inorder, elem+1, inEnd,
 indexes);
 return root;
 }

 TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
 map<int,int> indexes;
 for (int i = 0; i < inorder.size(); i++){
 indexes[inorder[i]] = i;
 }
 int n = preorder.size();
 int m = inorder.size();
 TreeNode* root = constructTree(preorder, 0, n-1, inorder, 0, m-1, indexes);
 return root;
 }
};
```

## Serialize and Deserialize Binary Tree

<https://leetcode.com/problems/serialize-and-deserialize-binary-tree/description/>

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

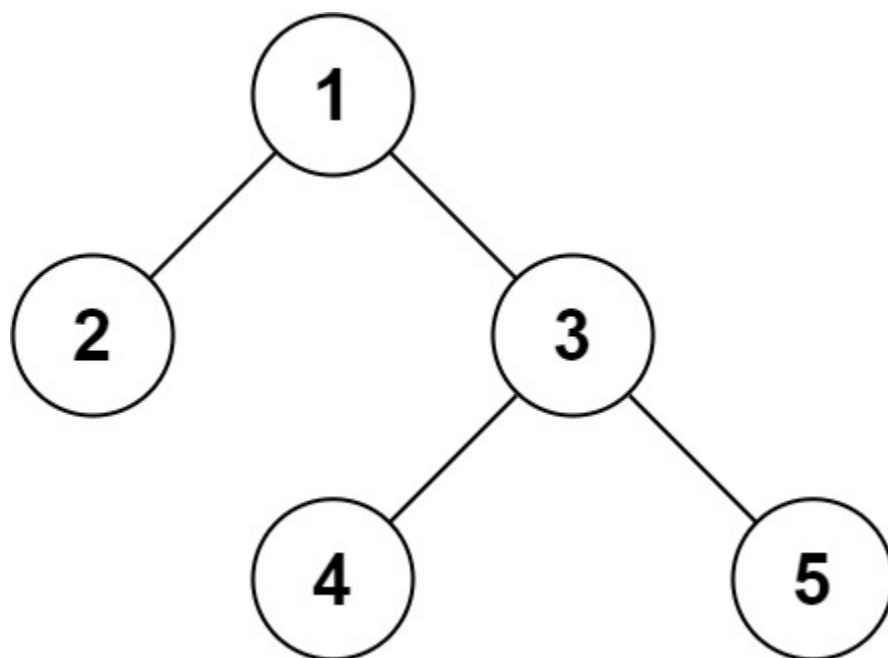
Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how



your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

**Clarification:** The input/output format is the same as [how LeetCode serializes a binary tree](#). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

**Example 1:**



**Input:** root = [1,2,3,null,null,4,5]

**Output:** [1,2,3,null,null,4,5]

**Example 2:**

**Input:** root = []

**Output:** []

```

class Codec {
private:
 void serializeHelper(TreeNode* root, std::string& result) {
 if (!root) {
 result += "null,";
 return;
 }
 result += std::to_string(root->val) + ",";
 serializeHelper(root->left, result);
 serializeHelper(root->right, result);
 }

 TreeNode* deserializeHelper(std::vector<std::string>& nodes, int& index) {
 if (index >= nodes.size() || nodes[index] == "null") {
 index++;
 return nullptr;
 }
 TreeNode* node = new TreeNode(std::stoi(nodes[index++]));
 node->left = deserializeHelper(nodes, index);
 node->right = deserializeHelper(nodes, index);
 return node;
 }
public:
 // Encodes a tree to a single string.
 std::string serialize(TreeNode* root) {
 std::string result;
 serializeHelper(root, result);
 return result;
 }

 // Decodes your encoded data to tree.
 TreeNode* deserialize(std::string data) {
 std::vector<std::string> nodes;
 std::string token;
 std::istringstream tokenStream(data);
 while (std::getline(tokenStream, token, ',')) {
 nodes.push_back(token);
 }
 int index = 0;
 return deserializeHelper(nodes, index);
 }
}

```

```
};
```

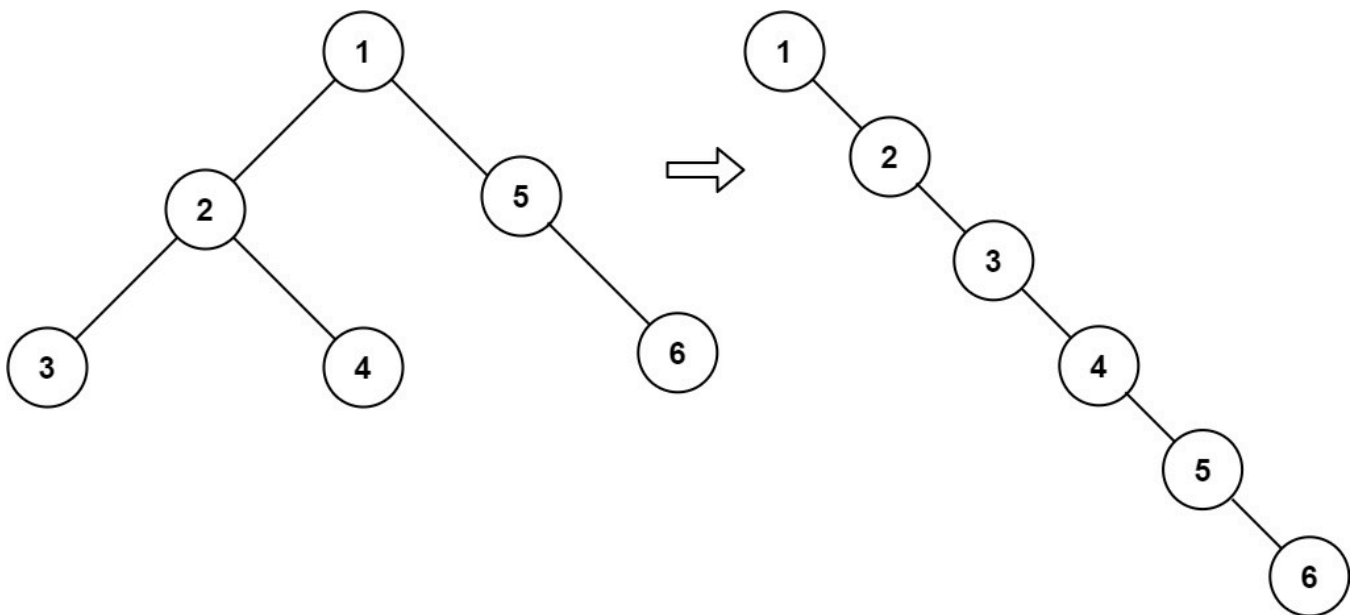
## Flatten A binary tree into a linked list

<https://leetcode.com/problems/flatten-binary-tree-to-linked-list/description/>

Given the `root` of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the `right` child pointer points to the next node in the list and the `left` child pointer is always `null`.
- The "linked list" should be in the same order as a **pre-order traversal** of the binary tree.

**Example 1:**



**Input:** root = [1,2,5,3,4,null,6]

**Output:** [1,null,2,null,3,null,4,null,5,null,6]

**Example 2:**

**Input:** root = []

**Output:** []

**Example 3:**

**Input:** root = [0]

**Output:** [0]

```

class Solution {
public:
 void flatten(TreeNode *root)
 {
 while(root)
 {
 if(root->left)
 {
 TreeNode *temp = root->right;
 TreeNode *temp2 = root->left;
 while (temp2->right)
 {
 temp2 = temp2->right;
 }
 root->right = root->left;
 temp2->right = temp;
 }
 root->left = NULL;
 root=root->right;
 }
 }
};

```

# Binary Search Tree

## Ceil in a binary search tree

<https://www.geeksforgeeks.org/problems/implementing-ceil-in-bst/1>

Given a BST and a number **X**, find **Ceil of X**.

**Note:** Ceil(X) is a number that is either equal to X or is immediately greater than X.

If Ceil could not be found, return -1.

### Example 1:

**\*\*Input:**

**\*\* 5**

```
/
1 7
```

```
2
```

```
3
```

X = 3

**Output:** 3

**Explanation:** We find 3 in BST, so ceil of 3 is 3.

**Example 2:**

**\*\*Input:**

**\*\* 10**

```
/
5 11
```

```
/ \
```

```
4 7
```

```
8
```

X = 6

**Output:** 7

**Explanation:** We find 7 in BST, so ceil of 6 is 7.

**Your task:**

You don't need to read input or print anything. Just complete the function **findCeil()** to implement ceil in BST which returns the ceil of **X** in the given **BST**.

```

int findCeil(Node* root, int input) {
 if (root == NULL) return -1;
 if (root->data < input) return findCeil(root->right,input);
 if (root->data >= input){
 int lef = findCeil(root->left,input);
 if (lef == -1) return root->data;
 else return lef;
 }
}

```

## Floor in a binary search tree

<https://www.geeksforgeeks.org/problems/floor-in-bst/1>

You are given a BST(Binary Search Tree) with **n** number of nodes and value **x**. your task is to find the greatest value node of the BST which is smaller than or equal to x.

**Note:** when x is smaller than the smallest node of BST then returns -1.

### Example:

#### Input:

n = 7 2

81

/

42 87

\

66 90

/

45

x = 87

#### Output:

87

#### Explanation:

87 is present in tree so floor will be 87.

### Example 2:

#### Input:

n = 4 6

8

/

7 9

x = 11

**Output:**

9

### Your Task:-

You don't need to read input or print anything. Complete the function **floor()** which takes the integer **n** and BST and integer x returns the floor value.

```
int floor(Node* root, int x) {
 // Code here
 if (root == nullptr) {
 return -1;
 }
 if (root->data <= x){
 int rig = floor(root->right,x);
 if (rig == -1) return root->data;
 else return rig;
 }
 if (root->data > x){
 return floor(root->left,x);
 }
}
```

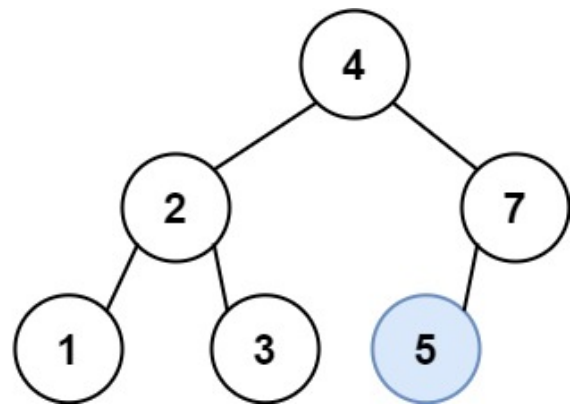
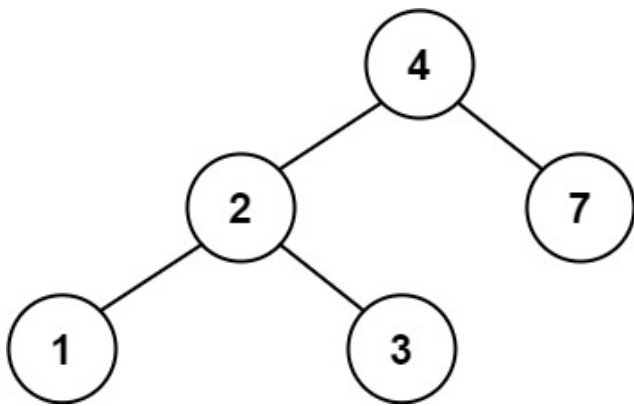
## Insert into a binary tree

<https://leetcode.com/problems/insert-into-a-binary-search-tree/description/>

You are given the `root` node of a binary search tree (BST) and a `value` to insert into the tree. Return *the root node of the BST after the insertion*. It is **guaranteed** that the new value does not exist in the original BST.

**Notice** that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return **any of them**.

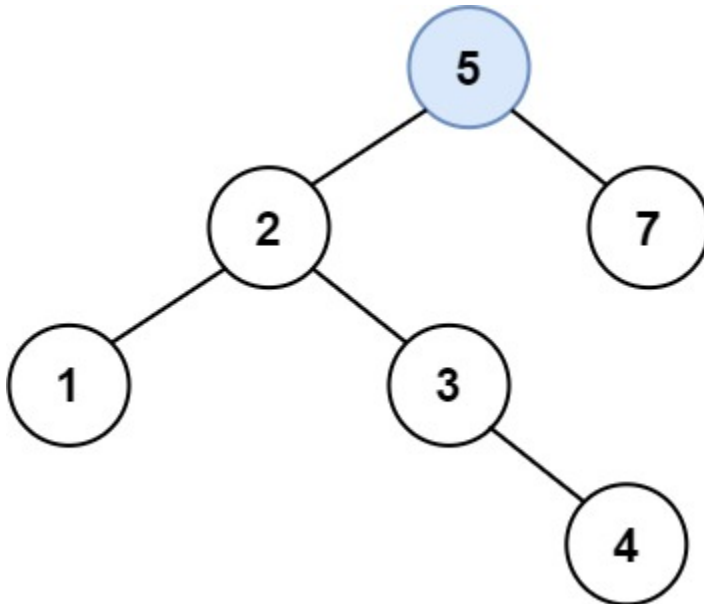
**Example 1:**



**Input:** root = [4,2,7,1,3], val = 5

**Output:** [4,2,7,1,3,5]

**Explanation:** Another accepted tree is:



**Example 2:**

**Input:** root = [40,20,60,10,30,50,70], val = 25

**Output:** [40,20,60,10,30,50,70,null,null,25]

**Example 3:**

**Input:** root = [4,2,7,1,3,null,null,null,null,null], val = 5

**Output:** [4,2,7,1,3,5]



```

private:
 void insertIntoBSTHelper(TreeNode* root, int val) {
 if (root == nullptr){
 TreeNode* newNode = new TreeNode(val);
 root = newNode;
 return;
 }
 if (root->val >= val){
 if (root->left == nullptr){
 TreeNode* newNode = new TreeNode(val);
 root->left = newNode;
 return;
 }
 else
 insertIntoBST(root->left, val);
 }
 else{
 if (root->right == nullptr){
 TreeNode* newNode = new TreeNode(val);
 root->right = newNode;
 return;
 }
 insertIntoBST(root->right, val);
 }
 }
public:
 TreeNode* insertIntoBST(TreeNode* root, int val) {
 if (root == nullptr) return new TreeNode(val);
 insertIntoBSTHelper(root, val);
 return root;
 }
}

```

## Delete from a binary tree

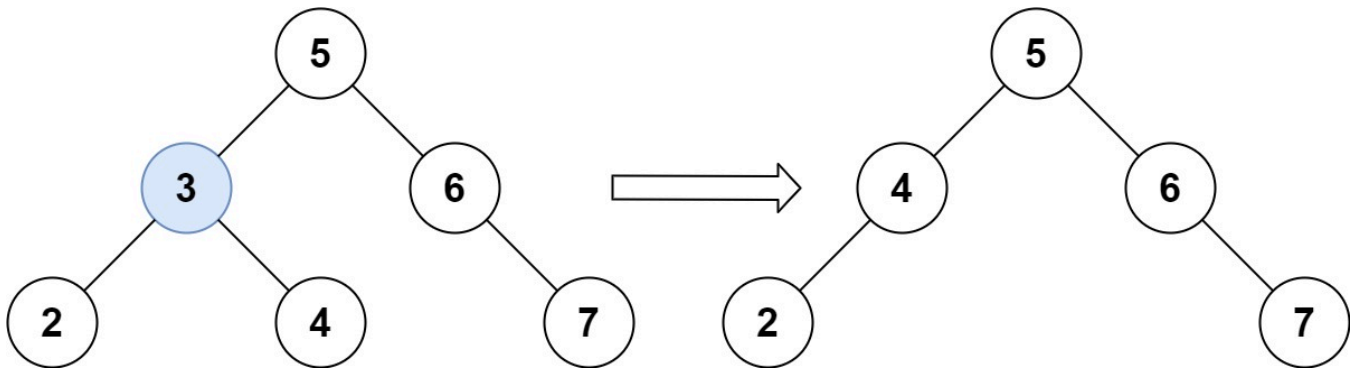
<https://leetcode.com/problems/delete-node-in-a-bst/description/>

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return *the root node reference (possibly updated) of the BST*.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

**Example 1:**



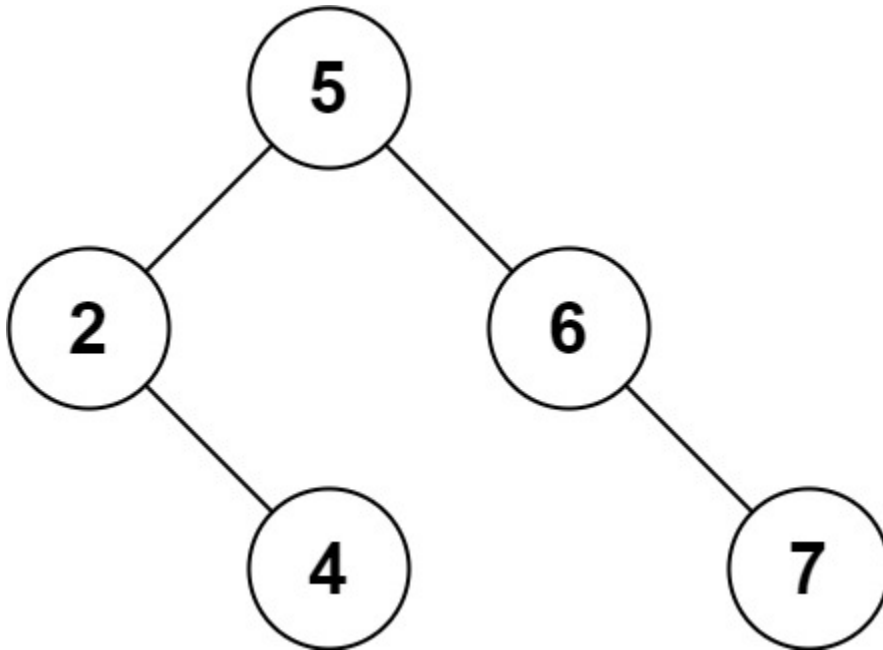
**Input:** root = [5,3,6,2,4,null,7], key = 3

**Output:** [5,4,6,2,null,null,7]

**Explanation:** Given key to delete is 3. So we find the node with value 3 and delete it.

One valid answer is [5,4,6,2,null,null,7], shown in the above BST.

Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.



**Example 2:**

**Input:** root = [5,3,6,2,4,null,7], key = 0

**Output:** [5,3,6,2,4,null,7]

**Explanation:** The tree does not contain a node with value = 0.

### Example 3:

**Input:** root = [], key = 0

**Output:** []

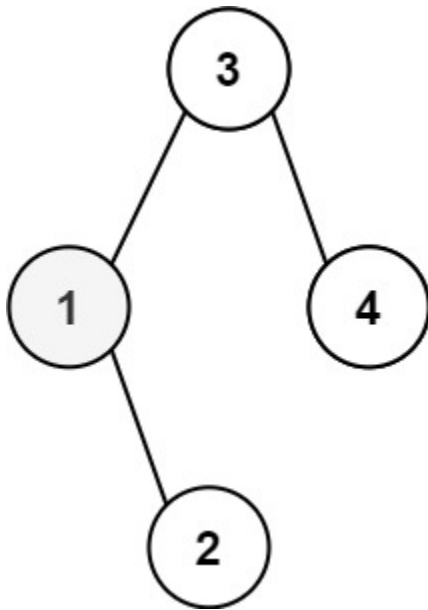
```
public:
 TreeNode* findMin(TreeNode* node) {
 while (node->left != nullptr) {
 node = node->left;
 }
 return node;
 }
 TreeNode* deleteNode(TreeNode* root, int key) {
 if (root == nullptr) {
 return root;
 }
 if (key < root->val) {
 root->left = deleteNode(root->left, key);
 } else if (key > root->val) {
 root->right = deleteNode(root->right, key);
 } else {
 // Node to be deleted found
 if (root->left == nullptr) {
 TreeNode* temp = root->right;
 delete root;
 return temp;
 } else if (root->right == nullptr) {
 TreeNode* temp = root->left;
 delete root;
 return temp;
 }
 // Node with two children: Get the inorder successor (smallest in the right subtree)
 TreeNode* temp = findMin(root->right);
 root->val = temp->val;
 root->right = deleteNode(root->right, temp->val);
 }
 return root;
 }
}
```

# Find Kth Smallest in a BST

<https://leetcode.com/problems/kth-smallest-element-in-a-bst/description/>

Given the `root` of a binary search tree, and an integer `k`, return *the `k`th smallest value (1-indexed) of all the values of the nodes in the tree.*

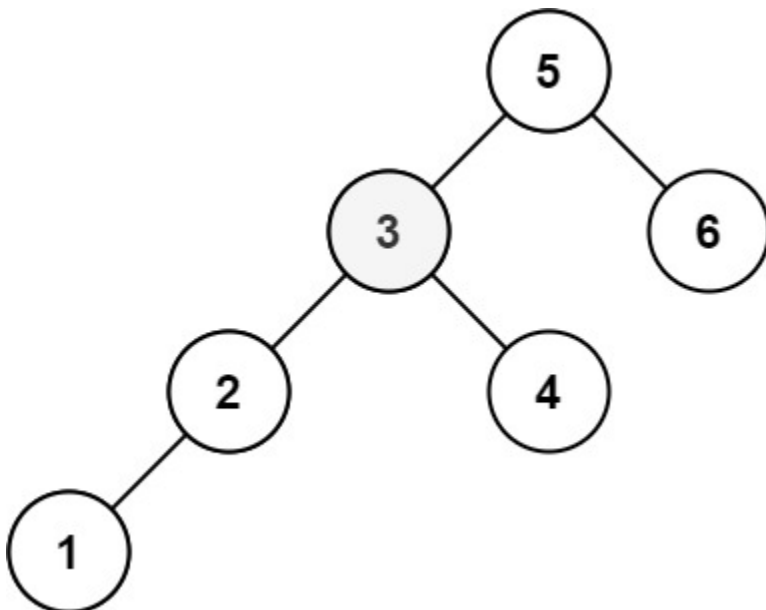
**Example 1:**



**Input:** `root = [3,1,4,null,2]`, `k = 1`

**Output:** 1

**Example 2:**



**Input:** root = [5,3,6,2,4,null,null,1], k = 3

**Output:** 3

```
int kthSmallest(TreeNode* root, int k) {
 stack<TreeNode*> st;
 while (!st.empty() || root != nullptr){
 // add all the nodes in the left, and keep pushing them in the stack
 //they will also act as our starting points
 while (root != nullptr){
 st.push(root);
 root = root->left;
 }
 root = st.top();
 st.pop();
 k--;
 if (k == 0){
 return root->val;
 }
 root = root->right;
 }
 return -1;
}
```

## Check if a tree is BST or not

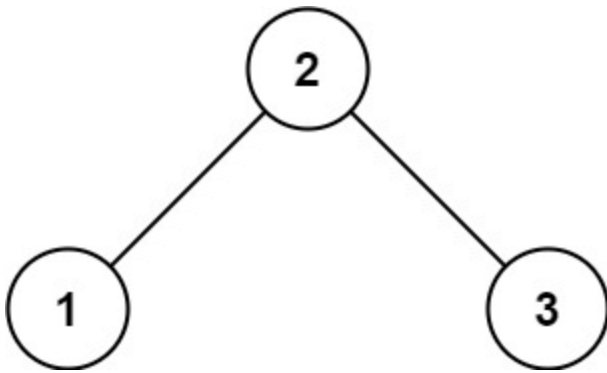
<https://leetcode.com/problems/validate-binary-search-tree/description/>

Given the `root` of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

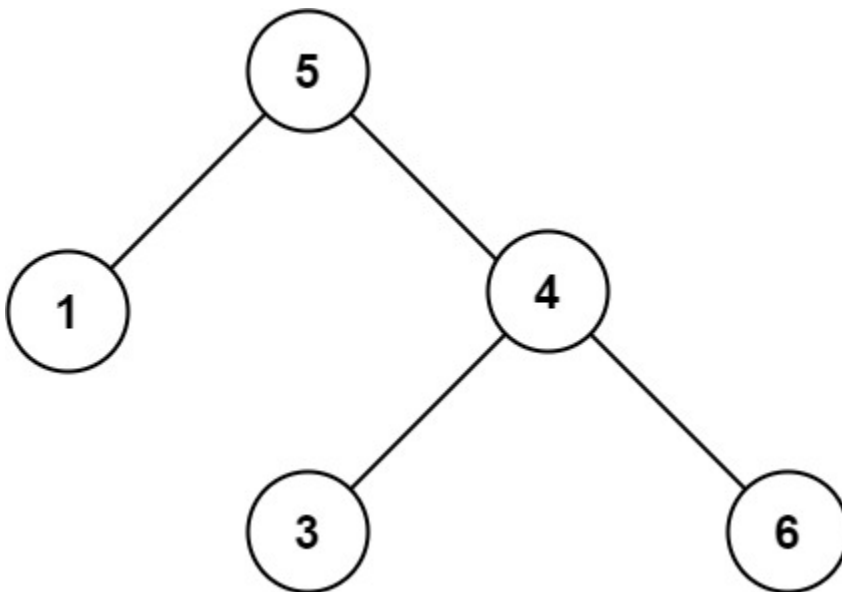
**Example 1:**



**Input:** root = [2,1,3]

**Output:** true

**Example 2:**



**Input:** root = [5,1,4,null,null,3,6]

**Output:** false

**Explanation:** The root node's value is 5 but its right child's value is 4.

```

bool isValidBST(TreeNode* root) {
 if (!root) return true;
 bool left = true;
 bool right = true;
 if (root->left) {
 if (root->left->val < root->val){
 left = isValidBST(root->left);
 }
 else return false;
 }
 if (root->right){
 if (root->right->val > root->val){
 right = isValidBST(root->right);
 }
 else return false;
 }
 return left && right;
}

```

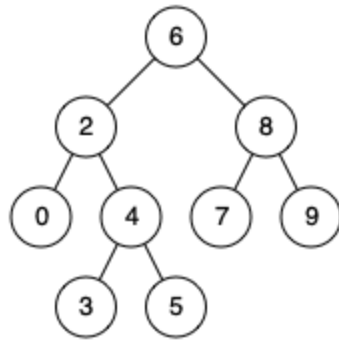
## LCA of a BST Node

<https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/description/>

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes **p** and **q** as the lowest node in **T** that has both **p** and **q** as descendants (where we allow **a node to be a descendant of itself**).”

**Example 1:**

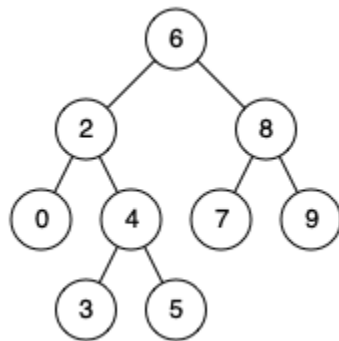


**Input:** root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

**Output:** 6

**Explanation:** The LCA of nodes 2 and 8 is 6.

**Example 2:**



**Input:** root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

**Output:** 2

**Explanation:** The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

**Example 3:**

**Input:** root = [2,1], p = 2, q = 1

**Output:** 2



```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
 // Base case
 if (!root) return nullptr;
 // If both p and q are less than root, then LCA lies in left subtree
 if (root->val > p->val && root->val > q->val) {
 return lowestCommonAncestor(root->left, p, q);
 }
 // If both p and q are greater than root, then LCA lies in right subtree
 if (root->val < p->val && root->val < q->val) {
 return lowestCommonAncestor(root->right, p, q);
 }
 // If root is split point, return root
 return root;
}

```

## Construct BST from preorder traversal

<https://leetcode.com/problems/construct-binary-search-tree-from-preorder-traversal/>

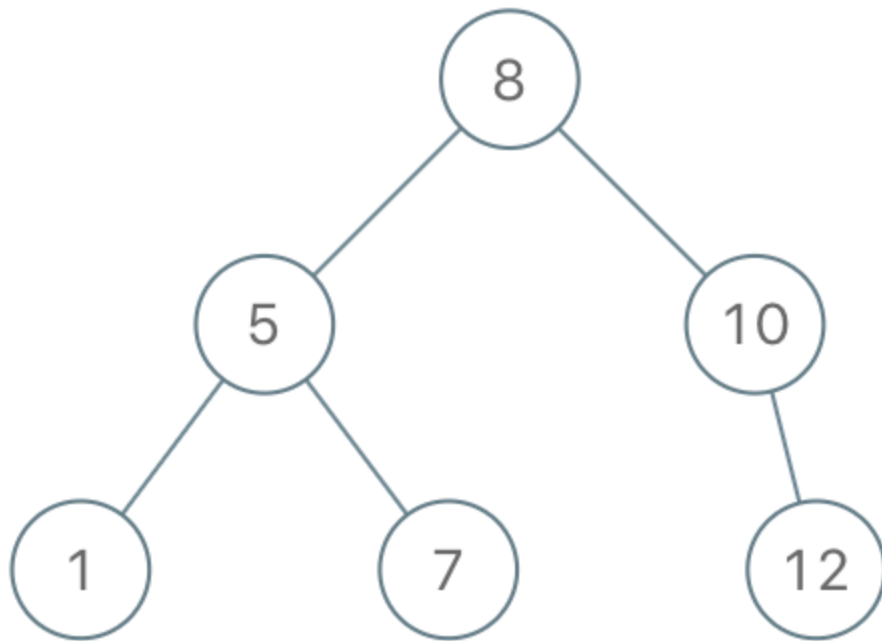
Given an array of integers preorder, which represents the **preorder traversal** of a BST (i.e., **binary search tree**), construct the tree and return *its root*.

It is **guaranteed** that there is always possible to find a binary search tree with the given requirements for the given test cases.

A **binary search tree** is a binary tree where for every node, any descendant of `Node.left` has a value **strictly less than** `Node.val`, and any descendant of `Node.right` has a value **strictly greater than** `Node.val`.

A **preorder traversal** of a binary tree displays the value of the node first, then traverses `Node.left`, then traverses `Node.right`.

**Example 1:**



**Input:** preorder = [8,5,1,7,10,12]

**Output:** [8,5,10,1,7,null,12]

### Example 2:

**Input:** preorder = [1,3]

**Output:** [1,null,3]

```
TreeNode* bestieHelper(vector<int>& pre, int &index, int bound){
 if (index == pre.size() || pre[index] > bound) return nullptr;
 int value = pre[index++];
 TreeNode* root = new TreeNode(value);
 root->left = bestieHelper(pre, index, value);
 root->right = bestieHelper(pre, index, bound);
 return root;
}

public:
 TreeNode* bstFromPreorder(vector<int>& preorder) {
 int index = 0;
 return bestieHelper(preorder, index, INT_MAX);
 }
}
```

# Inorder Successor

<https://www.geeksforgeeks.org/problems/inorder-successor-in-bst/>

[1?itm\\_source=geeksforgeeks&itm\\_medium=article&itm\\_campaign=bottom\\_sticky\\_on\\_article](https://www.geeksforgeeks.org/problems/inorder-successor-in-bst/1?itm_source=geeksforgeeks&itm_medium=article&itm_campaign=bottom_sticky_on_article)

Given a BST, and a reference to a Node x in the BST. Find the Inorder Successor of the given node in the BST.

## Example 1:

**Input:**

2

/

1 3

K(data of x) = 2

**Output:** 3

**Explanation:**

Inorder traversal : 1 2 3

Hence, inorder successor of 2 is 3.

## Example 2:

**\*\*Input:**

**\*\* 20**

/

8 22

/

4 12

/

10 14

K(data of x) = 8

**Output:** 10

**Explanation:**

Inorder traversal: 4 8 10 12 14 20 22

Hence, successor of 8 is 10.

## Your Task:

You don't need to read input or print anything. Your task is to complete the

function **inOrderSuccessor()**. This function takes the root node and the reference node as argument and returns the node that is inOrder successor of the reference node. If there is no successor, return null value.

```
Node* inOrderSuccessor(Node* root, Node* x) {
 // Step 1: If the node has a right child, the successor is the leftmost node of the right subtree.
 if (x->right) {
 Node* R = x->right;
 while (R->left) R = R->left;
 return R;
 }

 // Step 2: If the node doesn't have a right child, the successor is one of its ancestors.
 Node* successor = nullptr;
 Node* ancestor = root;
 while (ancestor != x) {
 if (x->data < ancestor->data) {
 successor = ancestor; // so far this is the deepest node for which x is in the left subtree.
 ancestor = ancestor->left;
 } else {
 ancestor = ancestor->right;
 }
 }
 return successor;
}
```

## Binary Search Tree Iterator

<https://leetcode.com/problems/binary-search-tree-iterator/description/>

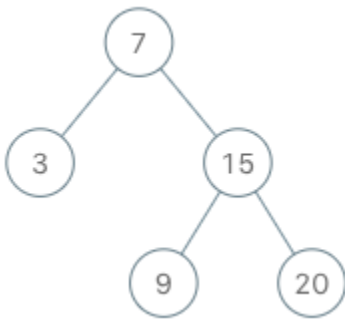
Implement the `BSTIterator` class that represents an iterator over the **in-order traversal** of a binary search tree (BST):

- `BSTIterator(TreeNode root)` Initializes an object of the `BSTIterator` class. The `root` of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- `boolean hasNext()` Returns `true` if there exists a number in the traversal to the right of the pointer, otherwise returns `false`.
- `int next()` Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to `next()` will return the smallest element in the BST.

You may assume that `next()` calls will always be valid. That is, there will be at least a next number in the in-order traversal when `next()` is called.

### Example 1:



### Input

```
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next", "hasNext", "next", "hasNext"]
```

```
[7, 3, 15, null, null, 9, 20, [], [], [], [], [], [], [], [], []]
```

### Output

```
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

### Explanation

```
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
```

```
bSTIterator.next(); // return 3
```

```
bSTIterator.next(); // return 7
```

```
bSTIterator.hasNext(); // return True
```

```
bSTIterator.next(); // return 9
```

```
bSTIterator.hasNext(); // return True
```

```
bSTIterator.next(); // return 15
```

```
bSTIterator.hasNext(); // return True
```

```
bSTIterator.next(); // return 20
```

```
bSTIterator.hasNext(); // return False
```

```

class BSTIterator {
 void putInStack(TreeNode* root, stack<TreeNode*>& inOrder){
 TreeNode* temp = root;
 while (temp != nullptr){
 inOrder.push(temp);
 temp = temp->left;
 }
 }

public:
 stack<TreeNode*> inOrder;
 BSTIterator(TreeNode* root) {
 putInStack(root, inOrder);
 }

 int next() {
 TreeNode* cur = inOrder.top();
 inOrder.pop();
 if (cur->right != nullptr) putInStack(cur->right, inOrder);
 return cur->val;
 }

 bool hasNext() {
 if (inOrder.size()) return true;
 else return false;
 }
}

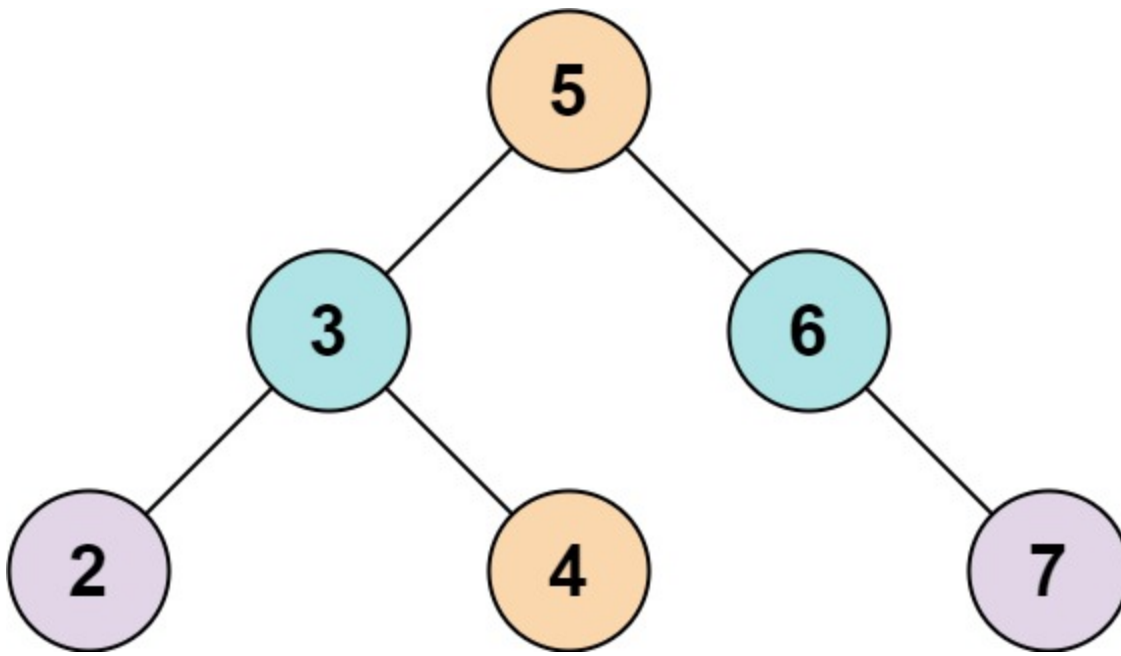
```

## Two Sum in a BST

<https://leetcode.com/problems/two-sum-iv-input-is-a-bst/description/>

Given the `root` of a binary search tree and an integer `k`, return `true` if there exist two elements in the BST such that their sum is equal to `k`, or `false` otherwise.

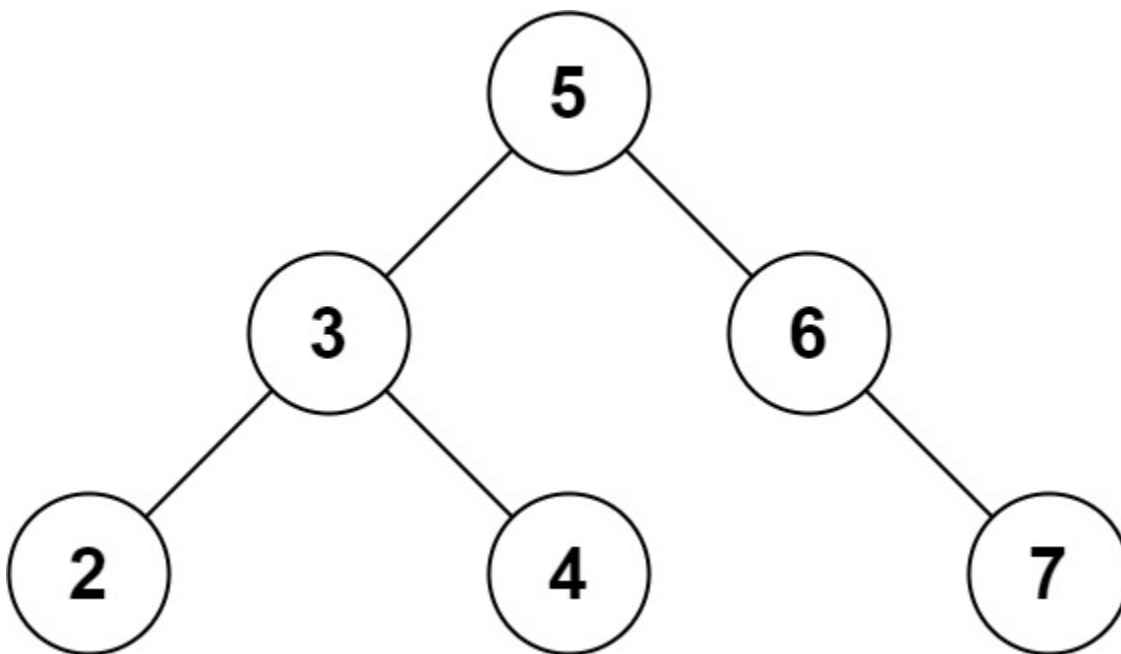
**Example 1:**



**Input:** root = [5,3,6,2,4,null,7], k = 9

**Output:** true

**Example 2:**



**Input:** root = [5,3,6,2,4,null,7], k = 28

**Output:** false

```

bool findTargetHelper(TreeNode* root, int k, std::unordered_set<int>& seen) {
 if (root == nullptr) return false;
 if (seen.count(k - root->val)) return true;
 seen.insert(root->val);
 return findTargetHelper(root->left, k, seen) || findTargetHelper(root->right, k, seen);
}

bool findTarget(TreeNode* root, int k) {
 unordered_set<int> seen;
 return findTargetHelper(root, k, seen);
}

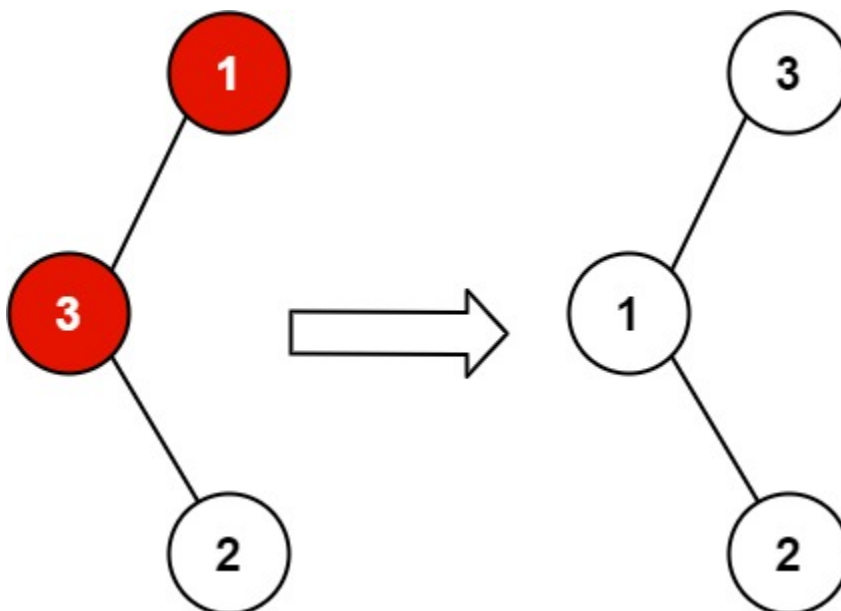
```

## Correct BST with two nodes swapped

<https://leetcode.com/problems/recover-binary-search-tree/description/>

You are given the `root` of a binary search tree (BST), where the values of **exactly** two nodes of the tree were swapped by mistake. *Recover the tree without changing its structure.*

**Example 1:**



**Input:** root = [1,3,null,null,2]

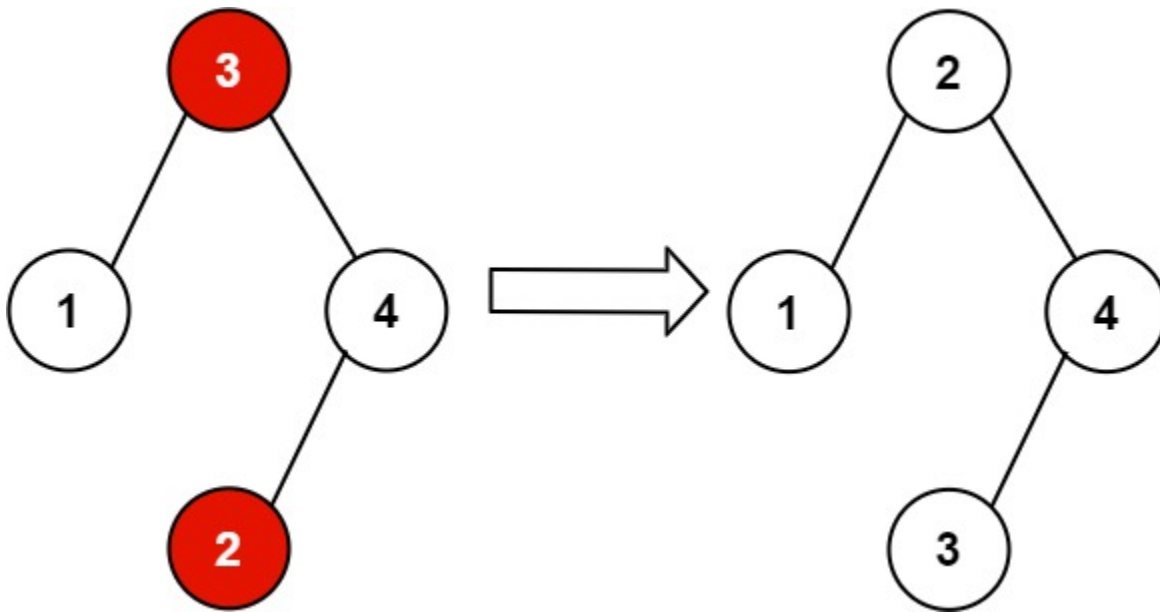
**Output:** [3,1,null,null,2]

**Explanation:** 3 cannot be a left child of 1 because  $3 > 1$ . Swapping 1 and 3 makes the BST



valid.

**Example 2:**



**Input:** root = [3,1,4,null,null,2]

**Output:** [2,1,4,null,null,3]

**Explanation:** 2 cannot be in the right subtree of 3 because  $2 < 3$ . Swapping 2 and 3 makes the BST valid.

**Constraints:**

- The number of nodes in the tree is in the range `[2, 1000]` .
- `-231 <= Node.val <= 231 - 1`

**Follow up:** A solution using `O(n)` space is pretty straight-forward. Could you devise a constant `O(1)` space solution?

```

void recoverTree(TreeNode* root) {
 TreeNode* first = nullptr;
 TreeNode* second = nullptr;
 TreeNode* prev = nullptr;
 // Helper function to perform in-order traversal
 function<void(TreeNode*)> inorder = [&](TreeNode* node) {
 if (!node) return;
 inorder(node->left);
 if (prev && prev->val > node->val) {
 if (!first) {
 first = prev;
 }
 second = node;
 }
 prev = node;
 inorder(node->right);
 };
 inorder(root);
 // Swap values of the two nodes to correct the tree
 if (first && second) {
 std::swap(first->val, second->val);
 }
}

```

## Largest BST in a tree

<https://www.geeksforgeeks.org/problems/largest-bst/1>

Given a binary tree. Find the size of its largest subtree that is a Binary Search Tree.

**Note:** Here Size is equal to the number of nodes in the subtree.

### Example 1:

**Input:**

```

1
/
4 4
/
6 8

```

**Output:** 1

**Explanation:** There's no sub-tree with size greater than 1 which forms a BST. All the leaf Nodes are the BSTs with size equal to 1.

### Example 2:

**Input:** 6 6 3 N 2 9 3 N 8 8 2

6

/

6 3

\ /

2 9 3

\ /

8 8 2 **Output:** 2

**Explanation:** The following sub-tree is a BST of size 2:

2

/ \

N 8

### Your Task:

You don't need to read input or print anything. Your task is to complete the function

**largestBst()** that takes the root node of the Binary Tree as its input and returns the size of the largest subtree which is also the BST. If the complete Binary Tree is a BST, return the size of the complete Binary Tree.

```

public:
 /*You are required to complete this method */
 // Return the size of the largest sub-tree which is also a BST
 struct Info {
 int size; // Size of the subtree
 int min; // Minimum value in the subtree
 int max; // Maximum value in the subtree
 bool isBST; // Is the subtree a BST
 };

 Info largestBSTUtil(Node* root, int &maxSize) {
 // Base case: an empty tree is a BST of size 0
 if (root == nullptr) {
 return {0, INT_MAX, INT_MIN, true};
 }

 // Recursively get info of left and right subtrees
 Info leftInfo = largestBSTUtil(root->left, maxSize);
 Info rightInfo = largestBSTUtil(root->right, maxSize);

 Info curr;
 curr.size = 1 + leftInfo.size + rightInfo.size;
 curr.min = std::min(root->data, leftInfo.min);
 curr.max = std::max(root->data, rightInfo.max);

 // Current node's subtree is a BST if left and right subtrees are BSTs
 // and current node's data is greater than max of left subtree
 // and less than min of right subtree
 if (leftInfo.isBST && rightInfo.isBST && root->data > leftInfo.max && root->data < rightInfo.min) {
 curr.isBST = true;
 maxSize = std::max(maxSize, curr.size);
 } else {
 curr.isBST = false;
 }

 return curr;
 }

 int largestBst(Node* root) {

```

```
int maxSize = 0;
largestBSTUtil(root, maxSize);
return maxSize;
}
```